

Colophon	Modify in Place
NOTE	Copy of Modify
<-	tracemem()
List	lobstr()
Data frames	Garbage Collector

The object is not copied; its contents are modified where they are in memory	A listing of R and add-on packages, along with their versions, used to make a document
R copies an object and then modifies specific elements	Environments use reference semantics meaning that they try to modify in place as much as possible
Allows you to see when an object gets copied	Assignment operator in R
Provides diagnostic tools for inspecting memory usage and representation of R objects	Stores references to values, not the values themselves
Deletes unbound objects	Lists of vectors

Attributes	str()
Factors	NOTE
Four primary types of atomic vectors	Scalar
c()	NOTE
What does adding a class attribute to a vector do?	list()

Tells you what kind of data structure the object is	Metadata attached to R objects and stored in a named list
Lists are recursive meaning that they can contain other lists	S3 atomic vectors (integers, actually) that have a 'class' attribute set to 'factor' and a 'levels' attribute. They are used to represent/store categorical and ordinal data in R
Individual numeric value	Logical, Integer, Double, Character
Attributes are ephemeral, meaning that they are lost by most operations. For a vector the only two attributes routinely preserved are names and dim	Creates vectors
Used to create a list	Makes it an S3 object

Data Frame/Tibble	Data Frames vs. Tibbles
NULL	NA
Three subsetting operators in R	Six things that can be used to subset a vector
drop = FALSE	How if ifelse() different than a standard if-else statement in R
switch()	Three common pitfalls of for loops

• Tibbles never coerce their input (meaning they implement lazy evaluation) • Tibbles do not automatically transform non-syntactic names • Tibbles only recycle vectors of length 1 • Tibbles allow references to other already created variables during construction	S3 vectors built on top of lists
Represents missing or unknown values in R	• Special R type that is always length 0 and can't have any attributes • Often used to represent an empty or absent vector
• Positive integers • Negative integers • Logical vector • Names • Nothing • Zero	• <code>[</code> • <code>[[</code> • <code>\$</code>
<code>ifelse()</code> is vectorized	Used to avoid simplifying an array or data frame into a simpler array
1. Growing vectors: always preallocate memory for vectors 2. Iterating over <code>1:length(x)</code>: always use <code>seq_along(x)</code> 3. Losing attributes with S3 objects	Useful when several possible actions might be taken based on the evaluation of a single expression

condition of an if-else statement	while loop
repeat	Formals of a function
Name masking	on_exit()
Infix function	Function components
Anonymous function	>

Performs the action while the condition is TRUE	Should always evaluate to a single TRUE or FALSE , otherwise it will generate an error
The formal arguments; the arguments a function is defined to have	Repeats the action forever (or until it encounter break)
• Specifies a chunk of code to run whenever a function is exited (terminates) for whatever reason • An exit handler	Having different values bound to the same name in different environments
• The formals(), the list of arguments that control how you call the function • The body(), the code inside the function • The environment(), the data structure that determines how the function finds the values associated with the names	• A function of two arguments and specified between its two arguments • Examples: +, %in%
Pipe operator in R	A function without a name

Scoping	Lexical Scoping
Name Masking	Functions versus Variables
A Fresh Start	Dynamic Lookup
Lazy Evaluation	Promise
Variadic Function	NOTE

- Looks up the values of names based on how the function is defined, not how it is called - Follows four primary rules - Name masking - Functions versus variables - A fresh start - Dynamic lookup - R uses lexical scoping	The act of finding the value associated with a name
When you use a name in a function call, R ignores non-function objects when looking for that value	- Names defined inside a function mask names defined outside a function - If a name isn't defined inside a function, R looks one level up
R looks for values when the function is run, not when the function is created	Every time a function is called, a new environment is created to host its execution
Has three components: - An expression which gives rise to the delayed computation - An environment where the expression should be evaluated - A value which is computed and cached the first time a promise is accessed when the expression is evaluated in the specified environment	Function arguments are only evaluated when they are accessed
We prefer implicit function returns since explicit function returns use <code>return()</code>	- A function that can take a varying ber of arguments - Utilizes <code>...</code>

Function Forms	NOTE
An environment is	What is the enclosing/parent environment of a function?
What is the execution environment of a function?	<code>rlang::env()</code>
NOTE	<code><<-</code>
Function Environment	Closure

The environment is the data structure that powers scoping	• prefix: The function name comes before its arguments. Constitutes the majority of function calls in R • infix: The function names comes in between its arguments • replacement: Functions that replace values by assignment • special: Functions like <code>[]</code>, <code>if</code>, and <code>for</code>
• The environment in which the function is defined • The parent/enclosing environment of the execution environment spawned by the calling of the function	A named list
Allows for the creation of an environment	The environment spawned by the call of a function in which to perform the computations of the function
• Super assignment operator • Never creates a variable in the current environment, but instead modifies an existing variable found in a parent environment • If it doesn't find an existing variable, it will create one in the global environment	Every environment has a parent/enclosing environment
A function that encloses their environments	• The environment bound when the function is created • Used for lexical scoping

Namespaces	NOTE
NOTE	Execution Environment
Caller Environment	Frame
Call Stack	Dynamic Scoping
What is the difference between handling and signalling conditions?	Three most important types of conditions

• The package environment is the external interface to the package. It how the user finds a function in the attached package using <code>::</code> • The namespace environment is the internal interface to the package. It controls how we find the function.	Ensure that every package works the same way regardless of what packages are attached by the user
• Usually ephemeral ie once the function has completed, the environment will be garbage collected • Its parent is the function environment	Look at pages 160-161 to get a better idea of how environments work
Has three key components: • An expression giving the function call (which is what <code>traceback</code> prints out • An environment which is typically the execution environment of a function • A parent, which is the previous call in the call stack	The environment from which the function was called
Looking up variables in the calling stack rather than in the enclosing environment	• Made up of several frames • Visualized with <code>traceback()</code> or <code>lobstr::cst()</code>
1. Errors: Most severe; indicate that there is no way for a function to continue and execution must stop 2. Warnings: Typically indicate that something has gone wrong but the function has been able to at least partially recover 3. Messages: Mildest; inform users that some action has been performed on their behalf	• Signalling is about issuing the condition and is done by the function author • Handling is about managing what happens when a signal is sent and is done by the function user

Errors are thrown by	Ignoring Conditions
Condition Handlers	<code>tryCatch()</code>
<code>apply()</code>	Functional
<code>map()</code>	NOTE:
<code>map2()</code>	<code>walk()</code> functions

• Ignore errors with <code>try()</code> • Ignore warnings with <code>suppressWarnings()</code> • Ignore messages with <code>suppressMessages()</code>	• <code>stop()</code> • <code>rlang::abort()</code>
Typically used to handle error conditions	Allow the user to temporarily override or supplement the default behavior
A function that takes a function as an input and returns a vector as the output	• Applies a function to part (or all) of a matrix or array • NOTE: Never use <code>apply()</code> with a data frame as it always coerces to a matrix.
There are 23 variants of <code>map()</code>	• Takes a vector and a function and calls the function once for each element of the vector, and returns the results in a list. • Base R equivalent is <code>lapply()</code> • There are other versions. The most common is probably <code>map_dbl()</code> which returns a vector instead of a list.
Functions that ignore the return values of the <code>.f</code> and instead return the input vector(s) <code>.x</code> invisibly	Same as <code>map()</code> but takes two vectors instead of just one

<code>pmap()</code>	<code>reduce()</code>
<code>accumulate()</code>	Predicate Functional
Function Factory	The enclosing environment of the manufactured function is an execution environment of the function factory
Why are Function Factories and Maximum Likelihood Estimation a good pair?	<code><-</code> versus <code><<-</code>
Function Operator	<code>safely()</code>

• Takes of vector of length n and returns a vector of length 1 by calling a function with a pair of values at a time • Usually used to generalize a binary function to work with any number of inputs	• Same as <code>map()</code> but allows for any number of input vectors • Gives user much finer control over argument matching
A function that returns a single TRUE or FALSE	• Variant of <code>reduce()</code> • Returns the output at each intermediate step instead of just the final output
• Manufactured functions are enclosed by the creating function's execution environment. Symbols not explicitly defined in the body of the function will therefore be first looked for in that execution environment which was made "not ephemeral" by the returned function. • Since a single function factory can make many different functions, the use of "an" instead of "the" is particularly important.	A function that creates another function(s)
• <code><-</code> is the assignment operator and only creates bindings in the current environment. • <code><<-</code> is the superassignment operator and creates bindings in the parent environment of the current environment. If no such binding is found in the search path, the binding is created in the global environment.	For a given statistical model, the likelihood function is a function of the model and the data. Knowing the model, a function factory can therefore be created that accepts the data and return the likelihood function.
• Accepts a function and returns a version that will capture any conditions such as errors • Makes functional programming a bit easier when an error may occur	• A function that takes one (or more) functions as input and returns a function as the output. • In mathematics these functions are called transforms and in computing they may be called wrappers.

<code>memoise::memoise()</code>	<code>delay_by()</code>
Main R OOP Systems	Class
Method	Method Dispatch
<code>sloop package</code>	<code>sloop::otype()</code>
NOTE:	Generic (Generic Function)

A function operator that modifies the input function by adding a little <code>Sys.sleep()</code> time ahead of the evaluation of the function.	• Accepts a function <code>f()</code> and returns a function <code>mf()</code> that performs exactly as <code>f()</code> does with one key difference. Every time <code>mf()</code> is run, it records its inputs and outputs. • Each subsequent time <code>mf()</code> is run, it will look at the input arguments, and if they match a previous call, it will simply lookup the past result instead of reperforming the computation.
The type of an object	• S3 • R6 • S4
The process of finding the correct method given a class ie the selection of a method based on a generic and an object.	The implementation for a specific class
Tells you what kind of object (in the sense of OOP) an R object is	Useful for OOP stuff in R
• A function that accepts an object and then calls a method that corresponds to the generic which depends on the class of the input object. • The goal is to decouple the implementation (how the code works) from the interface (the required input type and output).	While only OO objects have a class attribute, every object has a base type

How do you create an S3 object?	<code>UseMethod()</code>
Inheritance	S3 Object
<code>sloop::ftype()</code>	Every class should have
Constructor (<code>new_myclass()</code>)	Validator (<code>validate_myclass()</code>)
Helper (<code>myclass()</code>)	Metaprogramming

Performs method dispatch	Assign it a class attribute with <code>class<-()</code>
A base type with at least a <code>class</code> attribute.	The nesting of classes, so that an object can have several classes of increasing particularity ie subclasses.
• A constructor function • A validator function • A helper function	Used to determine if a function is a generic
Performs more computationally expensive checks to ensure that the object has correct values	Efficiently creates new objects with the correct structure
The idea that code is data that can be inspected and modified programatically	Provides a convenient way for others to create objects of your class

Non-Standard Evaluation (NSE)	<code>rlang::expr()</code> versus <code>rlang::enexpr()</code>
Expression	Abstract Syntax Tree (AST)
<code>base::eval()</code>	<code>!!</code>
Debugging Approach: Make it repeatable	<code>traceback()</code>
<code>browser()</code>	Why is debugging RMarkdown/Quarto more challenging than ordinary debugging?

• <code>rlang::expr()</code> quotes whatever is passed to it directly • <code>rlang::enexpr()</code> is used inside functions by the function writer to capture the calling argument provided by the user, quoted	Used to describe the behavior of R functions
Code as a tree	Captured code
• Unquote Operator • Can be used to build complex code trees	• The primary tool for evaluating expressions • Takes two arguments: an expression and an environment
Shows the sequence of calls (the call stack) that lead to an error	Isolate the specific problem you're experiencing, preferably by producing a reproducible example
RMarkdown/Quarto work non-interactively in their own session, so they don't share the same workings as working in the typical interactive environment.	Interrupts an evaluation, giving you a shell into the current execution environment

Alternatives to browser()	Profiling
Microbenchmarking	bench::mark()

The process of measuring the runtime of each line of code or function call in the evaluation of some statement.	• Setting breakpoints in RStudio • Setting <code>option(error = recover)</code> • Using <code>debug()</code> and other related functions
• Used for microbenchmarking in R • Runs each expression at least once and at most enough times to take 0.5s • Returns the results as a tibble	A measurement of the performance of a very small piece of code